>>>>>>>>>>>>>>>>>>>>>>>>>Python<<<<<<<<<<<<<<<<<<<<<<<

```
import networkx as nx

import matplotlib.pyplot as plt


# Initialize the graph

G = nx.Graph()


# Add stations and lines (edges)

lines = {

        'Central': ['Notting Hill Gate', 'Queensway', 'Lancaster Gate', 'Marble Arch',

        'Bond Street'], 'Jubilee': ['Baker Street', 'Bond Street', 'Green Park',

        'Westminster', 'Waterloo'],

        'Victoria': ['Green Park', 'Oxford Circus', 'Warren Street', 'Euston', 'King’s Cross St

        Pancras'], 'Northern': ['Leicester Square', 'Tottenham Court Road', 'Goodge Street',

        'Warren Street', 'Euston'], 'Piccadilly': ['Green Park', 'Piccadilly Circus', 'Leicester

        Square', 'Covent Garden', 'Holborn']

      }
# Adding nodes and edges to the graph

for line, stations in lines.items():

        G.add_nodes_from(stations)

        G.add_edges_from([(stations[i], stations[i+1]) for i in range(len(stations) - 1)])  # Create edges


# Check if the graph is connected

  print("The graph is connected:", nx.is_connected(G))
```

```
# Position nodes using the spring layout
pos = nx.spring_layout(G)

# Draw the network
plt.figure(figsize=(12, 8))
for line, stations in lines.items():

# Find the index of the line for color mapping
line_index = list(lines.keys()).index(line)

# Draw edges first
nx.draw_networkx_edges(G, pos, edgelist=[(stations[i],
stations[i+1]) for i in range(len(stations)-1)], width=2,
alpha=0.5)

# Draw nodes
nx.draw_networkx_nodes(G, pos, nodelist=stations,
node_color=plt.cm.tab10(line_index), node_size=300,
label=line)

# Draw labels
nx.draw_networkx_labels(G, pos, labels={station: station
for station in stations}, font_size=9, font_color='black')

plt.title('London Tube Network Example')
plt.legend(scatterpoints=1)
    plt.axis('off') # Turn off the
    axis plt.show()
```

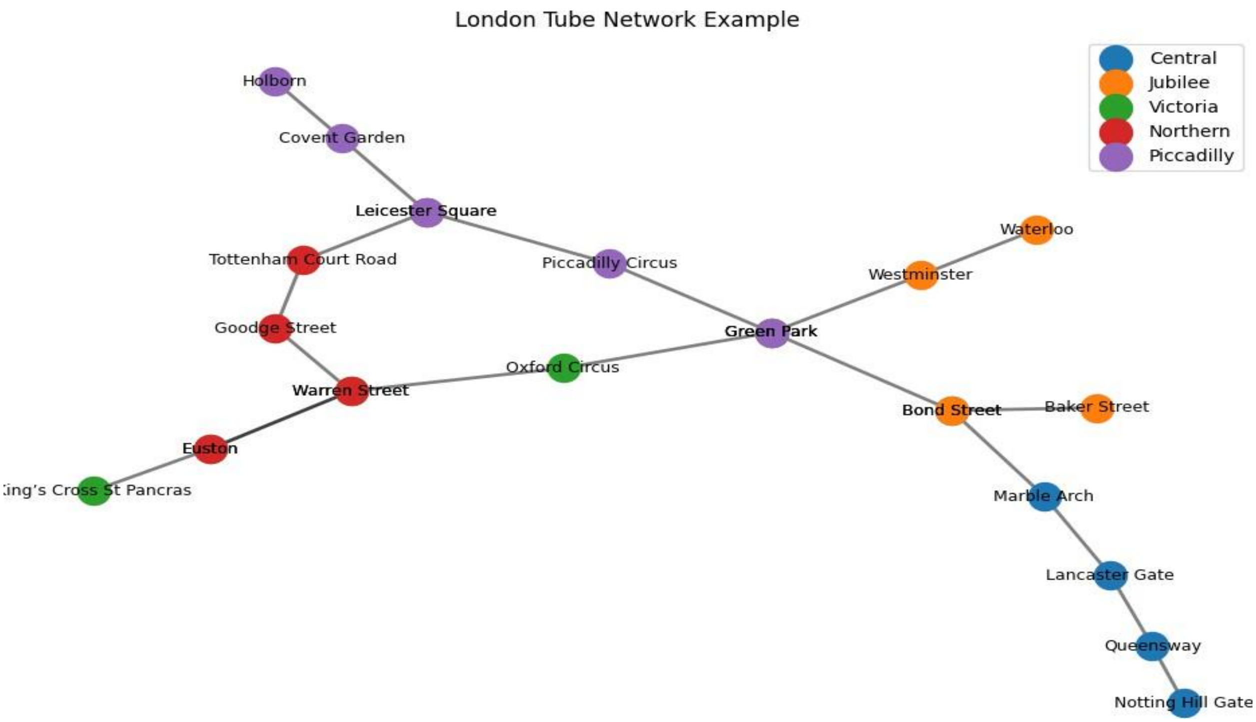

Figure 1 Visualisation of the underground network

>>>>>>>>>>>>>>>>>>>>>>>R<<<<<<<<<<<<<<<<<<<

Data used for this task: Excel spreadsheet “**cw_r.xlsx**” from Moodle (Coursework Submission Guidance section).

This spreadsheet contains multiple data tables (aka datasets), for example, ‘Table 3d’ describes different types of fraud and computer misuse in the year range of 2012 to 2021. Familiar yourself with the information recorded in each data table.

The analytic requirement is as follows:

1. Select and load the required dataset. Pre-process the dataset if necessary.
2. Generate a new data frame by sub-setting/slicing from the package **dplyr**, with this data frame representing only one category of fraud, **banking and credit industry fraud**.
3. Once the data frame of banking and credit industry fraud is defined. Visualise its dispersion and central tendency using ggplot2.

   The solution (code & plot) is as below.

```
# Load the dataset and convert it to long format for year-wise plotting
library(dplyr)
library(ggplot2)

# Loading the dataset

file_path <- "Data/Table_3d.xlsx"

df_banking<-read_excel(file_path, sheet = "Sheet1")
df_banking_long <- df_banking %>%
      pivot_longer(cols = `2012`:`2021`, names_to = "Year", values_to = "Count") %>
   % mutate(Year = as.factor(Year)) # Treat 'Year' as a categorical variable for plotting


# Plotting for 'Banking and credit industry fraud'

ggplot(df_banking_long, aes(x = Year, y = Count)) +
  geom_boxplot(fill = "lightblue") + geom_jitter(width =0.2,
  color = "darkblue") +
  ggtitle("Banking and Credit Industry Fraud Dispersion and Central Tendency") + xlab("Year")+
  ylab("Fraud Count") +
  theme_minimal()
```

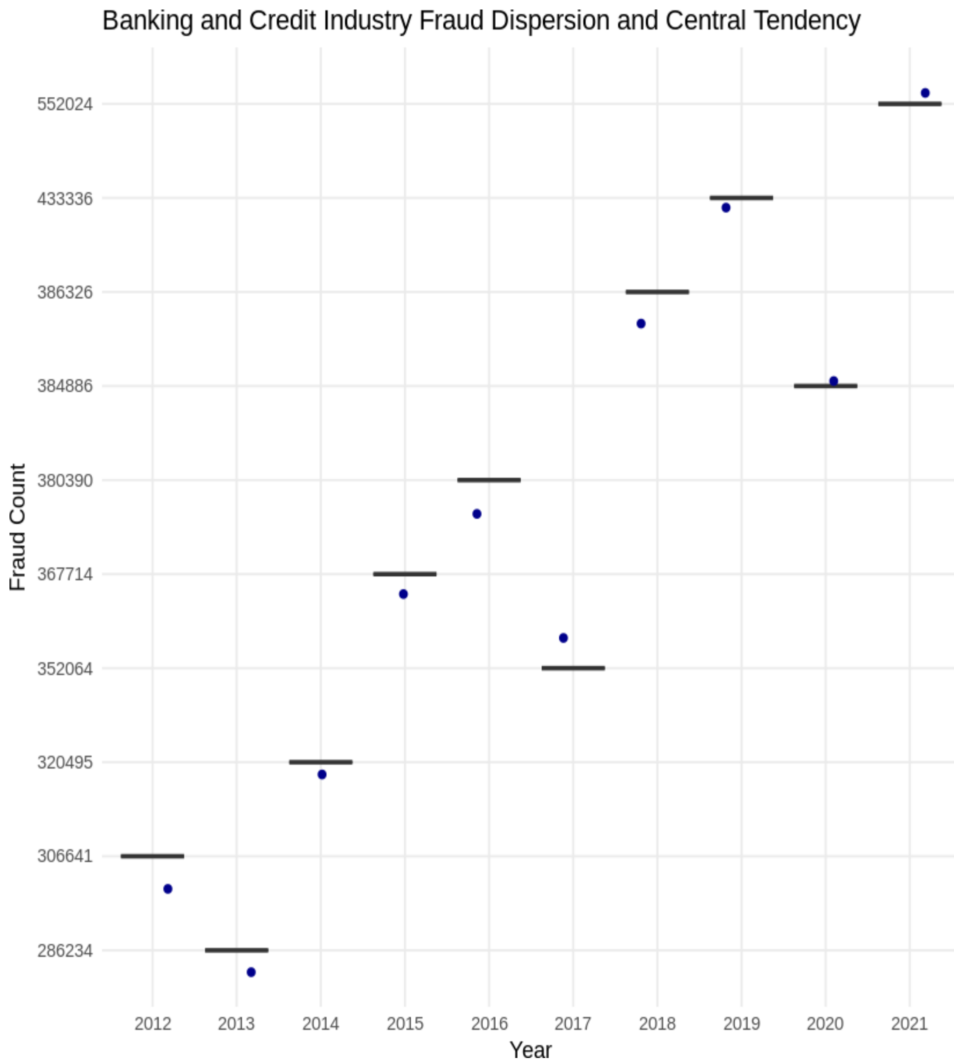

Figure 2 Banking and Credit Industry Fraud Dispersion and Central Tendency

Using same Excel spreadsheet “**cw_r.xlsx**”. We now check the regional trend/patterns recorded in Table 5.

The analytic requirement is as follows:
1. Load the required dataset and perform any necessary preprocessing.
2. Filter the dataset to include only records for England’s regions and counties.
3. Select an appropriate visualisation method (as taught in class) and create a plot showing the total offences across all nine regions of England. The plot should clearly highlight the regions with the highest and lowest offence count to support policy-making decisions.

The solution (code & plot) is as below.

```
# Load Table 5 directly, skipping introductory rows for clean data
access data_table5 <-read_excel("/Data/cw_r.xlsx", sheet = "Table 5",
skip = 5)

# Rename columns for clarity (adjust these names if the actual columns differ)
colnames(data_table5) <- c("Region", "County", "Total_Offences",
"Offence_Rate_per_1000", "Percentage_Change")


# Filter out any non-data rows and ensure numeric conversion for relevant
columns data_table5 <-data_table5 %>%
filter(!is.na(Region) & !grepl("Link|Source",
Region)) %>% mutate(Total_Offences
=as.numeric(Total_Offences),
      Offence_Rate_per_1000 = as.numeric(Offence_Rate_per_1000))


# Aggregate data by region for total offences and mean offence rate
region_summary <-data_table5 %>%
  group_by(Region) %>%
  summarise(Total_Offences = sum(Total_Offences, na.rm = TRUE),
    Mean_Offence_Rate =mean(Offence_Rate_per_1000, na.rm = TRUE))

  # Plot for Total Offences by Region
  ggplot(region_summary, aes(x = reorder(Region, -Total_Offences), y =
    Total_Offences))+ geom_bar(stat = "identity", fill = "steelblue") +
    coord_flip() +
  ggtitle("Total Offences by Region") +
  xlab("Region") + ylab("Total Offences") + theme_minimal()
```

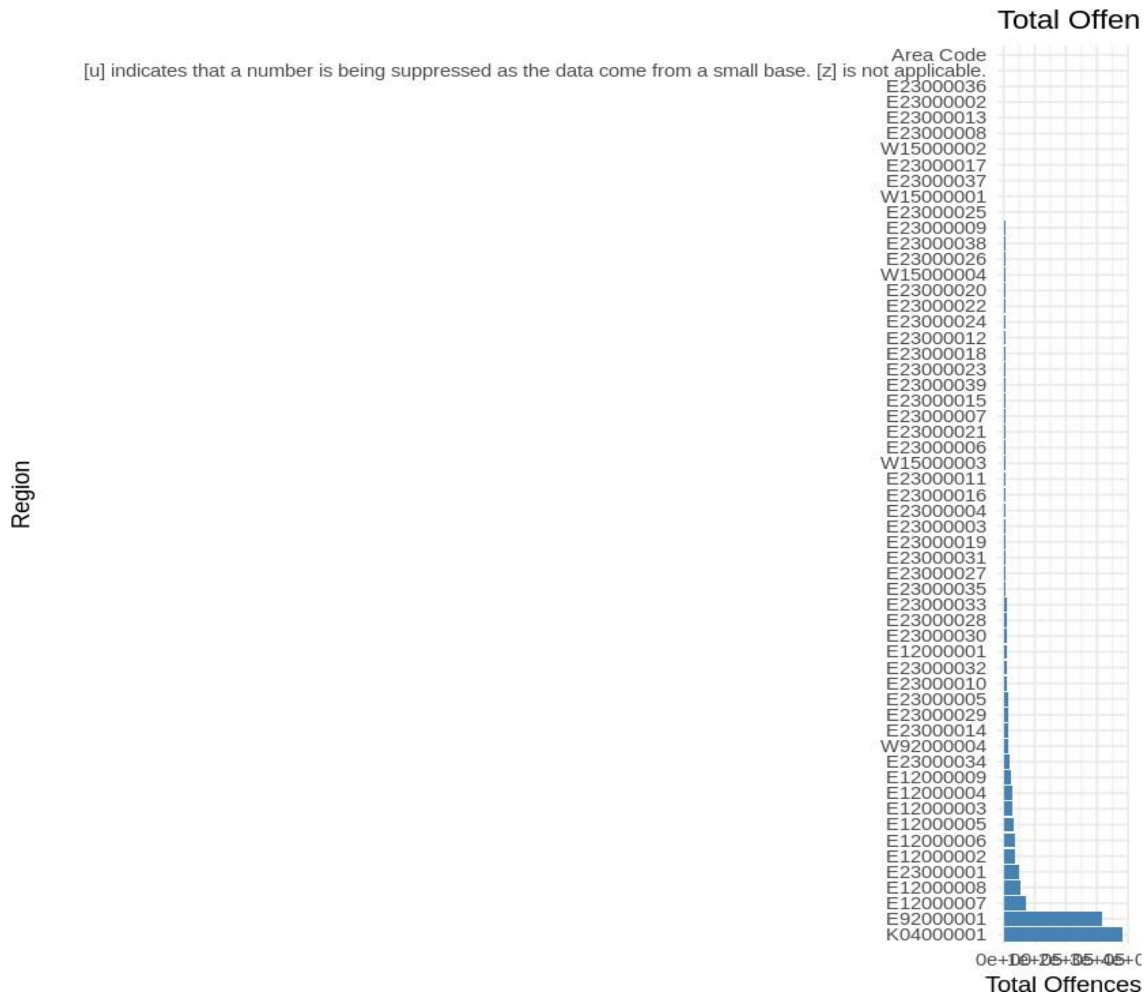

Figure 3 Total count of offence in England regions.